AMATH 582: BACKGROUND SUBTRACTION VIA LOW-RANK MATRIX APPROXIMATION: AN SVD APPROACH

PARTHA KRISHNAMURTHY

Applied And Computational Mathematics, University of Washington, Seattle, WA
ipartha2@uw.edu

1. Introduction and Overview

Removal of static background from moving objects in foreground of a streaming video is a popular application of Computer vision in many real world applications like fixed-camera surveillance or traffic monitoring systems. While the standard **Principal Component Analysis (PCA)** performed using a full **Singular Value Decomposition (SVD)** gives the best possible results, it is often preferred to perform a *randomized* PCA by employing *low-rank approximated* SVD. The precision requirement of the resultant background subtracted video is dependent on the application and in many cases low-precision and low-cost methods are preferred over moderately better accuracy results with prohibitively high computational costs. This approach exploits the high temporal correlation of the background (Low Rank) contrasted against the sparse, transient nature of foreground movements. In this paper we compare the full SVD implementation against the randomized SVD on video sequences with static backgrounds.

2. Theoretical Background

2.1. **Principal Component Analysis.** It is a common methodology in signal processing to decompose a signal as a linear combination of some basis functions. **Principal Component Analysis (PCA)** is a method of learning a set of basis functions from a given data set. It is a widely used method in data science for dimensionality reduction, data visualization, and feature extraction.
A dataset consisting of N data points denoted by $\{x_i\}_{i=1}^N$ in $\mathbb{R}^n$ can be represented as a matrix $\mathbf{X} \in \mathbb{R}^{n\times N}$. The mean $\hat{m}$ and variance $\hat{\Sigma}$ can be estimated as

$$\hat{m} = \frac{1}{N}\sum_{i=1}^{N} x_i, \quad \hat{\Sigma} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{m})(x_i - \hat{m})^T$$

The goal of PCA is to find a set of orthonormal basis vectors $\{u_i\}_{i=1}^k$ such that the projection of the data points on to this subspace maximizes the variance of the dataset.
PCA applies *eigen decomposition* to the sample covariance matrix of the dataset. The decomposition is given by,

$$\hat{\Sigma} = \hat{U}\hat{\Lambda}\hat{U}^T$$

where vectors $\hat{u_i}$ in matrix $\hat{U}$ are the principal components or eigenvectors and $\hat{\Lambda}$ is the diagonal matrix of eigenvalues $\lambda_i > 0$ in descending order.

Date: 02/13/2026.

2.1.1. *Rank-k approximation on PCA.* The eigenvectors $\hat{u_i}$ are the directions in $\mathbb{R}^n$ where the variance in data is maximum. The corresponding $\hat{\lambda_i}$ represents the amount of variance captured by this component. Selecting the first k columns of $\hat{U}$, yields the top k principal components. The optimal rank-k approximation of the covariance matrix is given by:

$$\hat{\Sigma_k} = \sum_{i=1}^{k} \hat{\lambda_i} \hat{u_i} \hat{u_i}^T$$

This approximation captures the maximum variance of the dataset feasible within a k-dimensional subspace.

2.2. **Singular Value Decomposition (SVD) for PCA.** The eigen decomposition of the covariance matrix is the core computational step of PCA. However computing the covariance matrix explicitly can be computationally expensive and numerically unstable. The PCA of a data set $X \in \mathbb{R}^{n \times N}$ is directly related to the **Singular Value Decomposition (SVD)** of the centered data matrix.

Let the centered data matrix, $\hat{X}$ defined as $\hat{X} = X - \hat{m}\mathbf{1}^T$ have a singular value decomposition given by,

$$\hat{X} = USV^T$$

where $U \in \mathbb{R}^{n \times n}$ and V $\in \mathbb{R}^{N \times N}$ are orthogonal matrices and S $\in \mathbb{R}^{n \times N}$ is a diagonal matrix with non-negative entries σ_i on the diagonal. Substituting this SVD decomposition into the expression for $\hat{\Sigma}$, we get

$$\hat{\Sigma} = \frac{1}{N}\hat{X}\hat{X}^T = \frac{1}{N}(USV^T)(USV^T)^T = \frac{1}{N}US\underbrace{V^TV}_{I}S^TU^T = \frac{1}{N}USS^TU^T$$

Since SS^T is a diagonal matrix with entries σ_i^2 on the diagonal, we identify that the eigenvalues of $\hat{\Sigma}$ are given by $\hat{\lambda_1} = \frac{\sigma_i^2}{N}$ and the corresponding eigenvectors are given by the columns of U.

2.3. **Randomized SVD.** The asymptotic *big-O* runtime of a deterministic *full*-SVD is $O(min(mn^2, m^2n))$. This computationally expensive operation can be replaced with a rank-k approximation by employing the method provided in Martinsson and Tropp, "Randomized numerical linear algebra: Foundations and algorithms", Acta Numerica, 2020.

The proposed algorithm consists of *two* stages:

- **Randomized Range Finder** Construct an orthonormal rank-k basis matrix Q, that approximates the range of data matrix X. This step is computationally efficient when $k \ll min(m, n)$.
- **Direct SVD** Project X onto the subspace spanned by Q to form a smaller matrix, then perform a deterministic SVD on this reduced matrix.

The detailed steps for this algorithm are provided in the next section.

2.4. **Subtracting Background from Videos.** A video with $n_x \times n_y$ resolution composed of n_f frames can be represented as an $(n_f \times (n_x \times n_y))$ 2-d matrix. The static background, being temporally consistent across all frames, will constitute as principal component and can be isolated by computing a low rank approximation. The foreground (moving objects) is then extracted by subtracting this low-rank background model from the original video matrix.

While the background is modeled as a rank-1 subspace, we compute the top k=20 singular vectors to ensure the dominant singular value is cleanly separated from the transient foreground dynamics, effectively preventing 'spectral leakage' of the moving objects into the background model.

3. Algorithm Implementation and Development

In this section we will detail the algorithms for the randomized SVD calculation and its subsequent usage in removing the static background on video sequences.

3.1. **Randomized Range Finder.** In order to find a computationally efficient approximate SVD of the data matrix X, we first compute a subspace that captures the low rank approximation to the range of X. We perform a QR factorization on a low rank sampled matrix that is generated by multiplying the data matrix X with a *Gaussian*. The orthonormal matrix Q captures the low rank approximation for the range of data matrix X.

Algorithm 1 Randomized Range Finder

Require: Data matrix X $\in \mathbb{R}^{m\times n}$, target rank k, oversampling parameter p
Ensure: Orthonormal matrix $Q \in \mathbb{R}^{m\times(k+p)}$
1: Initialize Gaussian random matrix, $W \in \mathbb{R}^{n\times(k+p)}$
2: Compute sample matrix, $Y = XW \in \mathbb{R}^{m\times(k+p)}$
3: Compute QR factorization, $Y = QR$
4: **return** Q

3.2. **Randomized SVD.** Projecting X on to the orthonormal basis computed in 1 generates a small matrix, $B \in \mathbb{R}^{(k+p)\times n}$. Mapping the left singular vectors obtained by performing SVD on the small matrix B on to the orthonormal matrix Q gives the approximate left singular vectors of X.

Algorithm 2 Randomized SVD

Require: Data matrix X $\in \mathbb{R}^{m\times n}$, matrix $Q \in \mathbb{R}^{m\times(k+p)}$ from 1
Ensure: Approximate rank-k SVD of X: U_k, Σ_k, V_k
1: Project X onto Q to form matrix, $B = Q^T X \in \mathbb{R}^{(k+p)\times n}$
2: Compute SVD of B, $B = \hat{U}\hat{\Sigma}\hat{V}^T$
3: Approximate the left singular vectors of X, $U = Q\hat{U}$
4: **return** $U, \hat{\Sigma}, \hat{V}^T$

3.3. **Background subtraction from video.** As mentioned in the Theoretical background section, the 3-dimensional video data is first transformed into a 2 dimensional $Time \times Space$ matrix. The randomized SVD described in 2 will then isolate the principal components across all frames. The isolated background matrix is then subtracted from the original video frame leaving behind the foreground information.

Algorithm 3 Subtracting static background

Require: Video data, X $\in \mathbb{R}^{n_f\times n_x\times n_y}$
Ensure: Background subtracted video data, $\hat{X} \in \mathbb{R}^{n_f\times n_x\times n_y}$
1: Reshape X to be 2-d matrix, $X_{reshaped} = X.reshape(n_f, n_x \times n_y)$
2: Get orthonormal basis for range of $X_{reshaped}$ using algorithm 1, Q = RandomizedRangeFinder($X_{reshaped}$)
3: Get the approximate SVD for $X_{reshaped}$ using algorithm 2, $U, \hat{\Sigma}, \hat{V}^T$ = RandomizedSVD($X_{reshaped}$, Q)
4: Extract Rank-1 Background, $X_{background} = \sigma_1 u_1 v_1^T$ (Using 1st component only)
5: Compute Residual, $X_{foreground} = X_{reshaped} - X_{background}$
6: **return** $X_{foreground}$

Note: $X_{foreground}$ returned from the algorithm 3 will be a 2 dimensional matrix which should be reshaped back to $n_f \times n_x \times n_y$ before converting to a video format.

4. Computational Results

4.1. **Evaluation of randomized SVD.** Before employing the randomized SVD algorithm 2 into our background removal process 3, we evaluate its performance by tuning hyperparameter, target rank k and oversampling parameter p.

A synthetic data matrix, T, was generated by sampling random functions from a Gaussian process. The generated covariance matrix was then provided as an input to the randomized SVD algorithm 2. The error norm was computed as follows,

$$Q = \text{RandomizedRangeFinder(T)}$$
$$\hat{U}, \hat{\Sigma}, \hat{V}^T = \text{RandomizedSVD(Q, T, k, p)}$$
$$error = \frac{norm(T - \hat{U} \cdot diag(\hat{\Sigma}) \cdot \hat{V}^T)}{\text{norm(T)}}$$

The error norm was computed *10* times against all combinations of oversampling parameters $p = [0, 5, 10]$ and target rank $k = [5, 10, 20]$. The mean and the standard deviation of the results in tabulated in 1.

Rank k	**Oversampling parameter p**	**Error Mean**	**Error Standard Deviation**
5	0	0.590	0.035
5	5	0.406	0.014
5	10	0.380	7×10^{-4}
10	0	0.201	0.046
10	5	0.075	0.002
10	10	0.070	5×10^{-4}
20	0	0.022	0.001
20	5	0.017	6×10^{-4}
20	10	0.015	6×10^{-4}

Table 1. Hyperparameter tuning results on the performance of Randomized SVD algorithm 2

The relative reconstruction error decreases monotonically with increasing rank k, achieving $\approx 1\%$ error at rank $k = 20$.

4.2. **Background subtraction results.** A video captured using a fixed-camera was used to validate the background removal process. The video file was decomposed into individual frames and was modeled as $Time \times Space$ matrix. The output of the algorithm, preserving the input dimensions, was reshaped back to original dimension so as to reconstruct a background subtracted video sequence.

We evaluate the performance using two metrics: hyperparameter sensitivity and computational efficiency against deterministic SVD.

4.2.1. *Hyperparameter sensitivity: Rank k and Oversampling p.* Although the static background is theoretically modeled as a rank-1 subspace, we compute a larger basis with different target ranks and oversampling parameters. This oversampling strategy creates a spectral buffer that ensures the dominant singular vector u_1 converges to the true static background without contamination from dynamic foreground elements, which typically reside in the subsequent components $u_2, \ldots, u_5$. By pushing the truncation threshold well beyond the energetic foreground modes, we maximize the spectral gap between the calculated subspace and the residual noise, thereby guaranteeing that the first principal component remains a pure representation of the static scene. A comparison on the effect of hyperparameter on the final result is shown in 1.

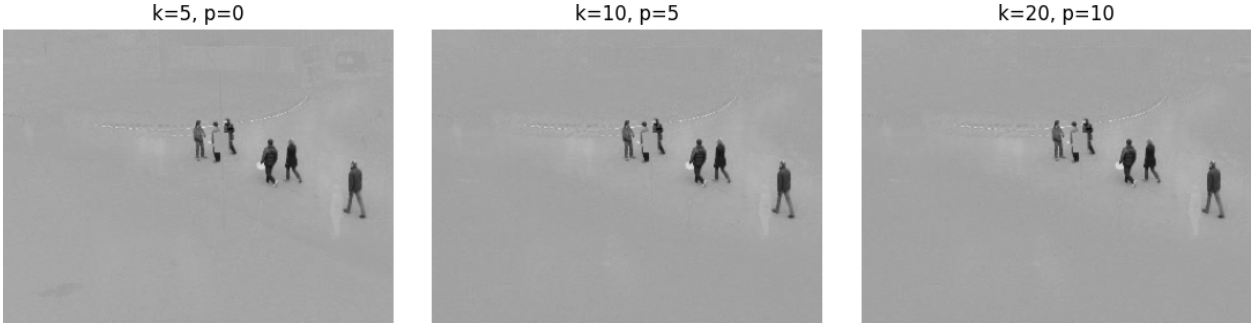


FIGURE 1. Impact of Randomized SVD hyperparameters on background removal. Higher rank ($k = 20$) effectively suppresses background leakage.

4.2.2. *Performance comparison: Randomized vs Full SVD.* The core engine of the background removal process is the extraction of *low-rank* temporal components by employing **SVD**. The background subtraction was performed using both the full SVD and the randomized SVD and the results were within pixel intensity variance of 15%. Figure 2 demonstrates that the background subtracted frames are visually indistinguishable.

Although the results are visibly similar the efficiency of the algorithms were significantly different. While the randomized SVD isolated the low rank components in < 0.5 seconds, the full SVD required 10 *sec*. This represents $\approx 20\times$ speed up. This confirms that while Randomized SVD introduces a slight approximation error, the order-of-magnitude reduction in computational cost makes it strictly superior for real-time video processing applications where latency is critical.



FIGURE 2. Qualitative comparison of background subtraction. The Randomized SVD achieves comparable visual quality to Full SVD at a fraction of the computational cost.

5. SUMMARY AND CONCLUSIONS

This paper investigated computationally efficient algorithms for utilizing **Principal Component Analysis (PCA)** to perform background subtraction in video sequences. Our approach relied on the hypothesis that temporally consistent background features can be effectively isolated as the *low-rank* principal components of the spatiotemporal data matrix.

We established that **Singular Value Decomposition (SVD)** provides a numerically stable alternative to traditional covariance-based eigen decomposition. Furthermore, we demonstrated that while deterministic Full SVD provides exact low-rank isolation, *Randomized SVD* achieves comparable visual fidelity at a fraction of the computational cost. The results confirm that Randomized SVD is a highly efficient solution, making it an optimal candidate for real-time video processing systems where latency is a critical constraint.